

Comprehensive Scheduling Algorithms Notes with Problems

Table of Contents

1. [Introduction to CPU Scheduling](#introduction)
2. [First Come First Serve (FCFS)](#fcfs)
3. [Shortest Job First (SJF)](#sjf)
4. [Round Robin (RR)](#round-robin)
5. [Priority Scheduling](#priority)
6. [Multilevel Queue Scheduling](#multilevel-queue)
7. [Multilevel Feedback Queue](#multilevel-feedback)
8. [Comparison and Selection Criteria](#comparison)

Introduction to CPU Scheduling {#introduction}

Why CPU Scheduling?

In a multiprogramming environment, multiple processes compete for CPU time. The CPU scheduler decides which process gets the CPU next and for how long.

Key Terms & Metrics:

- ##### 1. **Arrival Time (AT)****: When process arrives in ready queue
- ##### 2. **Burst Time (BT)****: Time needed by process for CPU execution
- ##### 3. **Completion Time (CT)****: When process finishes execution
- ##### 4. **Turnaround Time (TAT)****: Total time from arrival to completion

...

$TAT = Completion\ Time - Arrival\ Time$

...

- ##### 5. **Waiting Time (WT)****: Time process waits in ready queue

...

$WT = Turnaround\ Time - Burst\ Time$

...

- ##### 6. **Response Time (RT)****: Time from arrival to first CPU allocation

...

$RT = First\ CPU\ Time - Arrival\ Time$

...

Performance Metrics:

- **CPU Utilization****: Percentage of time CPU is busy
- **Throughput****: Number of processes completed per unit time
- **Average Turnaround Time****: $(Sum\ of\ all\ TAT) / Number\ of\ processes$
- **Average Waiting Time****: $(Sum\ of\ all\ WT) / Number\ of\ processes$

1. First Come First Serve (FCFS) {#fcfs}

Algorithm Understanding:

FCFS is like a **queue at a bank** - whoever arrives first gets served first.

Step-by-Step Process:

...

1. Arrange processes by arrival time
2. Execute processes in arrival order
3. Each process runs to completion (non-preemptive)
4. Next process starts only after current one finishes

...

Real-World Example:

Restaurant Service: Customers are served in the order they arrive, regardless of how much time they need to order.

Algorithm Implementation:

...

FCFS_Scheduling(processes[]):

1. Sort processes by arrival_time
2. current_time = 0
3. For each process p in sorted_processes:
 - a. If current_time < p.arrival_time:
current_time = p.arrival_time
 - b. p.start_time = current_time
 - c. p.completion_time = current_time + p.burst_time
 - d. current_time = p.completion_time
 - e. Calculate TAT, WT, RT for process p

...

🔥 Problem 1: Basic FCFS

****Given****:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

****Step-by-Step Solution****:

****Step 1****: Arrange by arrival time (already arranged)

****Step 2****: Create Gantt Chart

...

|--P1(8)--|--P2(4)--|--P3(9)--|--P4(5)--|
0 8 12 21 26

...

****Step 3**:** Calculate metrics for each process

****P1**:**

- Completion Time = 8
- TAT = $8 - 0 = 8$
- WT = $8 - 8 = 0$
- RT = $0 - 0 = 0$

****P2**:**

- Completion Time = 12
- TAT = $12 - 1 = 11$
- WT = $11 - 4 = 7$
- RT = $8 - 1 = 7$

****P3**:**

- Completion Time = 21
- TAT = $21 - 2 = 19$
- WT = $19 - 9 = 10$
- RT = $12 - 2 = 10$

****P4**:**

- Completion Time = 26
- TAT = $26 - 3 = 23$
- WT = $23 - 5 = 18$
- RT = $21 - 3 = 18$

****Results Table**:**

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	8	0	0	
P2	1	4	12	11	7	7
P3	2	9	21	19	10	10
P4	3	5	26	23	18	18

****Average TAT**** = $(8 + 11 + 19 + 23) / 4 = 15.25$

****Average WT**** = $(0 + 7 + 10 + 18) / 4 = 8.75$

🔥 Problem 2: FCFS with Different Arrival Times

****Given**:**

Process	Arrival Time	Burst Time
P1	2	3
P2	0	5
P3	4	2
P4	6	4

****Step-by-Step Solution**:**

****Step 1**:** Sort by arrival time: P2(0), P1(2), P3(4), P4(6)

****Step 2**:** Create Gantt Chart

...

|--P2(5)--|--P1(3)--|--P3(2)--|--P4(4)--|
0 5 8 10 14
...

****Step 3**:** Calculate metrics

****P2**:** CT=5, TAT=5-0=5, WT=5-5=0, RT=0-0=0

****P1**:** CT=8, TAT=8-2=6, WT=6-3=3, RT=5-2=3

****P3**:** CT=10, TAT=10-4=6, WT=6-2=4, RT=8-4=4

****P4**:** CT=14, TAT=14-6=8, WT=8-4=4, RT=10-6=4

****Average TAT**** = $(5 + 6 + 6 + 8) / 4 = 6.25$

****Average WT**** = $(0 + 3 + 4 + 4) / 4 = 2.75$

🔥 Problem 3: FCFS Convoy Effect

****Given**:**

Process	Arrival Time	Burst Time
P1	0	20
P2	1	2
P3	2	3

****Step-by-Step Solution**:**

****Gantt Chart**:**

...

|----P1(20)----|P2(2)|P3(3)|
0 20 22 25
...

****Calculations**:**

- P1: TAT=20, WT=0

- P2: TAT=22-1=21, WT=21-2=19

- P3: TAT=25-2=23, WT=23-3=20

****Average WT**** = $(0 + 19 + 20) / 3 = 13$

****Convoy Effect Demonstrated**:** Short processes (P2, P3) wait long time behind long process (P1).

FCFS Advantages & Disadvantages:

****Advantages**:**

- Simple to implement
- No starvation (all processes eventually execute)
- Fair in terms of arrival order

****Disadvantages**:**

- ****Convoy Effect****: Short processes wait behind long processes
- Poor average waiting time
- Non-preemptive (can't interrupt long processes)

2. Shortest Job First (SJF) {#sjf}

Algorithm Understanding:

SJF is like ****choosing the shortest checkout line**** at a supermarket - always pick the job that takes least time.

Types of SJF:

1. ****Non-Preemptive SJF****: Once a process starts, it runs to completion
2. ****Preemptive SJF (SRTF)****: If a shorter job arrives, preempt current job

Real-World Example:

****Emergency Room Triage****: Patients with quicker treatments are seen first (unless someone more urgent arrives).

Non-Preemptive SJF Algorithm:

...

SJF_NonPreemptive(processes[]):

1. current_time = 0
2. ready_queue = empty
3. While processes remain:
 - a. Add all processes with arrival_time <= current_time to ready_queue
 - b. If ready_queue empty: current_time = next_arrival_time
 - c. Select process with minimum burst_time from ready_queue
 - d. Execute selected process completely
 - e. Update current_time and calculate metrics

...

🔥 Problem 1: Non-Preemptive SJF

****Given****:

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

****Step-by-Step Solution**:**

****Time 0**:** Only P1 available → Execute P1

****Time 7**:** P1 completes. Available: P2(BT=4), P3(BT=1), P4(BT=4)
→ Select P3 (shortest)

****Time 8**:** P3 completes. Available: P2(BT=4), P4(BT=4)
→ Select P2 (same BT, P2 arrived first)

****Time 12**:** P2 completes. Available: P4(BT=4)
→ Execute P4

****Gantt Chart**:**

...

|--P1(7)--|P3(1)|--P2(4)--|P4(4)--|
0 7 8 12 16

...

****Calculations**:**

Process	AT	BT	CT	TAT	WT	RT
-----	----	----	----	----	----	----
P1	0	7	7	7	0	0
P2	2	4	12	10	6	6
P3	4	1	8	4	3	3
P4	5	4	16	11	7	7

****Average TAT**** = $(7 + 10 + 4 + 11) / 4 = 8$

****Average WT**** = $(0 + 6 + 3 + 7) / 4 = 4$

Preemptive SJF (SRTF) Algorithm:

...

SRTF_Algorithm(processes[]):

1. current_time = 0

2. While processes remain:

a. Check for new arrivals at current_time

b. If new arrival has shorter remaining time than current process:
Preempt current process

c. Execute shortest remaining time process for 1 unit

d. Update remaining times and current_time

...

🔥 Problem 2: Preemptive SJF (SRTF)

****Given**:**

Process	Arrival Time	Burst Time
-----	-----	-----

P1	0	8	
P2	1	4	
P3	2	9	
P4	3	5	

****Step-by-Step Solution**:**

****Time-by-Time Analysis**:**

****t=0**:** P1 starts (RT=8)
****t=1**:** P2 arrives (RT=4). $P2 < P1 \rightarrow$ Preempt P1, start P2
****t=2**:** P3 arrives (RT=9). P2(RT=3) is shortest $\rightarrow$ Continue P2
****t=3**:** P4 arrives (RT=5). P2(RT=2) is shortest $\rightarrow$ Continue P2
****t=5**:** P2 completes. Available: P1(RT=7), P3(RT=9), P4(RT=5) $\rightarrow$ Start P4
****t=10**:** P4 completes. Available: P1(RT=7), P3(RT=9) $\rightarrow$ Start P1
****t=17**:** P1 completes. Available: P3(RT=9) $\rightarrow$ Start P3
****t=26**:** P3 completes

****Detailed Timeline**:**

...

Time: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
 Proc: P1 P2 P2 P2 P2 P4 P4 P4 P4 P4 P1 P1 P1 P1 P1 P1 P1 P3 P3 P3 P3 P3 P3 P3 P3
 P3

...

****Gantt Chart**:**

...

|P1|--P2(4)--|P4(5)--|P1(7)--|P3(9)--|
 0 1 5 10 17 26

...

****Calculations**:**

Process	AT	BT	CT	TAT	WT	RT	
-----	----	----	----	----	----	----	
P1	0	8	17	17	9	0	
P2	1	4	5	4	0	0	
P3	2	9	26	24	15	15	
P4	3	5	10	7	2	2	

****Average TAT**** = $(17 + 4 + 24 + 7) / 4 = 13$

****Average WT**** = $(9 + 0 + 15 + 2) / 4 = 6.5$

🔥 Problem 3: SJF vs FCFS Comparison

****Given**** (Same processes for both algorithms):

Process	Arrival Time	Burst Time	
-----	-----	-----	
P1	0	6	

P2	1	2	
P3	2	8	
P4	3	3	

****FCFS Solution**:**

...

Gantt: |--P1(6)--|P2(2)|--P3(8)--|P4(3)|
0 6 8 16 19

Average WT = $(0 + 5 + 6 + 13) / 4 = 6$

...

****Non-Preemptive SJF Solution**:**

****Time 0**:** Execute P1 (only available)

****Time 6**:** Available: P2(2), P3(8), P4(3) → Choose P2

****Time 8**:** Available: P3(8), P4(3) → Choose P4

****Time 11**:** Available: P3(8) → Choose P3

...

Gantt: |--P1(6)--|P2(2)|P4(3)|--P3(8)--|
0 6 8 11 19

Average WT = $(0 + 5 + 9 + 5) / 4 = 4.75$

...

****Comparison**:** SJF reduces average waiting time from 6 to 4.75!

SJF Advantages & Disadvantages:

****Advantages**:**

- ****Optimal average waiting time**** for given set of processes
- Better than FCFS for short jobs
- Good throughput

****Disadvantages**:**

- ****Starvation**:** Long processes may never execute
- ****Prediction difficulty**:** Hard to predict actual CPU burst time
- Preemptive version has high context switching overhead

3. Round Robin (RR) {#round-robin}

Algorithm Understanding:

Round Robin is like ****passing a ball in a circle**** - each person gets the ball for a fixed time, then passes it to the next person.

Real-World Example:

****Time-sharing computer lab****: Each student gets 10 minutes on computer, then moves to back of queue.

Key Concept: Time Quantum (q)

- ****Small q****: More responsive, but high overhead
- ****Large q****: Approaches FCFS behavior
- ****Optimal q****: Usually 10-100ms in practice

Round Robin Algorithm:

...

RoundRobin(processes[], time_quantum):

1. ready_queue = processes sorted by arrival time
2. current_time = 0
3. While ready_queue not empty:
 - a. Get first process from ready_queue
 - b. If process remaining_time > time_quantum:
Execute for time_quantum
remaining_time -= time_quantum
Add process back to end of ready_queue
 - c. Else:
Execute for remaining_time
Process completes
 - d. Update current_time

...

🔥 Problem 1: Basic Round Robin

****Given****:

Process	Arrival Time	Burst Time
P1	0	10
P2	1	5
P3	2	8

****Time Quantum = 4****

****Step-by-Step Solution****:

****Initial Queue****: [P1]

****Round 1**** (t=0-4):

- Execute P1 for 4 units
- P1 remaining = $10 - 4 = 6$
- At t=1: P2 arrives, at t=2: P3 arrives
- Queue becomes: [P2, P3, P1]

****Round 2**** (t=4-8):

- Execute P2 for 4 units

- P2 remaining = $5-4 = 1$
- Queue becomes: [P3, P1, P2]

****Round 3**** (t=8-12):

- Execute P3 for 4 units
- P3 remaining = $8-4 = 4$
- Queue becomes: [P1, P2, P3]

****Round 4**** (t=12-16):

- Execute P1 for 4 units
- P1 remaining = $6-4 = 2$
- Queue becomes: [P2, P3, P1]

****Round 5**** (t=16-17):

- Execute P2 for 1 unit (remaining time)
- P2 completes
- Queue becomes: [P3, P1]

****Round 6**** (t=17-21):

- Execute P3 for 4 units (remaining time)
- P3 completes
- Queue becomes: [P1]

****Round 7**** (t=21-23):

- Execute P1 for 2 units (remaining time)
- P1 completes

****Gantt Chart****:

```

...
|P1(4)|P2(4)|P3(4)|P1(4)|P2(1)|P3(4)|P1(2)|
0   4   8   12  16  17  21  23
...

```

****Detailed Process Execution****:

Time	Process	Remaining After	Queue State
0-4	P1	6	[P2,P3,P1]
4-8	P2	1	[P3,P1,P2]
8-12	P3	4	[P1,P2,P3]
12-16	P1	2	[P2,P3,P1]
16-17	P2	0 (Complete)	[P3,P1]
17-21	P3	0 (Complete)	[P1]
21-23	P1	0 (Complete)	[]

****Calculations****:

Process	AT	BT	CT	TAT	WT	RT
P1	0	10	23	23	13	0

P2	1	5	17	16	11	3
P3	2	8	21	19	11	6

****Average TAT**** = $(23 + 16 + 19) / 3 = 19.33$

****Average WT**** = $(13 + 11 + 11) / 3 = 11.67$

🔥 Problem 2: Round Robin with Different Time Quantum

****Same processes, Time Quantum = 2****

****Step-by-Step Solution****:

****Execution Timeline****:

...

Time: 0-2: P1 (remaining=8), Queue: [P2,P3,P1]
 Time: 2-4: P2 (remaining=3), Queue: [P3,P1,P2]
 Time: 4-6: P3 (remaining=6), Queue: [P1,P2,P3]
 Time: 6-8: P1 (remaining=6), Queue: [P2,P3,P1]
 Time: 8-10: P2 (remaining=1), Queue: [P3,P1,P2]
 Time: 10-12: P3 (remaining=4), Queue: [P1,P2,P3]
 Time: 12-14: P1 (remaining=4), Queue: [P2,P3,P1]
 Time: 14-15: P2 (remaining=0), P2 completes, Queue: [P3,P1]
 Time: 15-17: P3 (remaining=2), Queue: [P1,P3]
 Time: 17-19: P1 (remaining=2), Queue: [P3,P1]
 Time: 19-21: P3 (remaining=0), P3 completes, Queue: [P1]
 Time: 21-23: P1 (remaining=0), P1 completes, Queue: []

...

****Gantt Chart****:

...

P1	P2	P3	P1	P2	P3	P1	P2	P3	P1	P3	P1	
0	2	4	6	8	10	12	14	15	17	19	21	23

...

****Results****:

Process	CT	TAT	WT
P1	23	23	13
P2	15	14	9
P3	21	19	11

****Average WT**** = 11 (same as q=4, but more context switches)

🔥 Problem 3: Round Robin Time Quantum Impact

****Given****:

Process	Arrival Time	Burst Time

P1	0	6	
P2	2	4	
P3	4	2	

****Comparing different time quantum**:**

****Time Quantum = 1**:**

...

Gantt: |P1|P1|P2|P3|P1|P2|P1|P3|P1|P2|P1|P2|
0 1 2 3 4 5 6 7 8 9 10 11 12

Average WT = 4, Context Switches = 11

...

****Time Quantum = 3**:**

...

Gantt: |P1(3)|P2(3)|P3(2)|P1(3)|P2(1)|
0 3 6 8 11 12

Average WT = 3, Context Switches = 4

...

****Time Quantum = 10** (Becomes FCFS):**

...

Gantt: |P1(6)|P2(4)|P3(2)|
0 6 10 12

Average WT = 2.67, Context Switches = 2

...

****Key Insight**:** Smaller time quantum gives better response time but higher overhead.

Time Quantum Selection Guidelines:

1. ****q = ∞****: Becomes FCFS
2. ****q → 0****: High context switching overhead
3. ****Optimal q****: 80% of processes should complete within one time quantum
4. ****Rule of thumb****: q should be slightly larger than typical CPU burst

Round Robin Advantages & Disadvantages:

****Advantages**:**

- ****Fair sharing****: Each process gets equal CPU time
- ****Good response time****: No process waits too long
- ****No starvation****: All processes eventually get CPU
- ****Suitable for time-sharing systems****

****Disadvantages**:**

- ****Context switching overhead****: Frequent switches
- ****Poor for I/O bound processes****: May waste quantum
- ****Average waiting time****: Usually higher than SJF

- **Time quantum selection**: Critical for performance

4. Priority Scheduling {#priority}

Algorithm Understanding:

Priority scheduling is like **hospital emergency room** - patients with higher priority (more critical condition) are treated first.

Types of Priority Scheduling:

1. **Non-Preemptive**: Higher priority process waits for current process to complete
2. **Preemptive**: Higher priority process immediately preempts lower priority process

Priority Assignment:

- **Higher number = Higher priority** (or vice versa, depends on system)
- **Static Priority**: Assigned at process creation, doesn't change
- **Dynamic Priority**: Changes based on process behavior

Real-World Examples:

- **Operating System processes** > **User processes**
- **System calls** > **Application code**
- **Interactive processes** > **Batch processes**

Non-Preemptive Priority Algorithm:

...

PriorityScheduling_NonPreemptive(processes[]):

1. current_time = 0
2. While processes remain:
 - a. Get all processes with arrival_time <= current_time
 - b. Select process with highest priority from available
 - c. Execute selected process completely
 - d. Update current_time and calculate metrics

...

🔥 Problem 1: Non-Preemptive Priority

Given (Higher number = Higher priority):

| Process | Arrival Time | Burst Time | Priority |

	-----	-----	-----	-----
P1	0	4	2	
P2	1	3	1	
P3	2	1	4	
P4	3	5	3	

Step-by-Step Solution:

Time 0: Only P1 available → Execute P1 (Priority=2)

****Time 4**:** P1 completes

Available processes: P2(Priority=1), P3(Priority=4), P4(Priority=3)

Highest priority: P3(4) → Execute P3

****Time 5**:** P3 completes

Available processes: P2(Priority=1), P4(Priority=3)

Highest priority: P4(3) → Execute P4

****Time 10**:** P4 completes

Available processes: P2(Priority=1)

Execute P2

****Gantt Chart**:**

...

|--P1(4)--|P3(1)|--P4(5)--|P2(3)|

0 4 5 10 13

...

****Calculations**:**

Process	AT	BT	Priority	CT	TAT	WT	RT
---------	----	----	----------	----	-----	----	----

P1	0	4	2	4	4	0	0
----	---	---	---	---	---	---	---

P2	1	3	1	13	12	9	9
----	---	---	---	----	----	---	---

P3	2	1	4	3	1	0	0
----	---	---	---	---	---	---	---

P4	3	5	3	8	5	0	0
----	---	---	---	---	---	---	---

****Average TAT**** = $(10 + 12 + 1 + 5) / 4 = 7$

****Average WT**** = $(6 + 9 + 0 + 0) / 4 = 3.75$

🔥 Problem 3: Priority Scheduling with Aging

****Problem**:** Prevent starvation using aging technique

****Given**:**

Process	Arrival Time	Burst Time	Initial Priority
---------	--------------	------------	------------------

P1	0	8	1
----	---	---	---

P2	1	2	5
----	---	---	---

P3	2	4	3
----	---	---	---

P4	3	1	4
----	---	---	---

****Aging Rule**:** Every 2 time units, increase priority of waiting processes by 1

****Step-by-Step Solution with Aging**:**

****Time 0-1**:** Execute P1 (only available)

****Time 1**:** P2 arrives with priority 5 > P1's priority 1 → Preempt P1, start P2

****Time 2****: P3 arrives (priority 3), P1 aged to priority 2. P2 still highest → Continue P2

****Time 3****: P2 completes, P4 arrives (priority 4)

Available: P1(priority 2), P3(priority 3), P4(priority 4)

Execute P4 (highest priority)

****Priority Aging Timeline****:

...

Time	P1 Priority	P2 Priority	P3 Priority	P4 Priority	Action
0	1	-	-	-	P1 starts
1	1	5	-	-	P2 preempts P1
2	1	5	3	-	P2 continues
3	2 (aged)	-	3	4	P4 starts (P2 complete)
4	2	-	3	4	P4 completes
5	3 (aged)	-	4 (aged)	-	P3 starts (higher priority)

...

****Final Execution Order****: P1(1) → P2(2) → P4(1) → P3(4) → P1(7)

Priority Inversion Problem:

****Scenario****: High priority process waits for resource held by low priority process

...

High Priority (P3) —waits for—> Resource <—held by— Low Priority (P1)

Medium Priority (P2) —preempts— P1, preventing P3 from getting resource

...

****Solution****: Priority Inheritance Protocol

- Low priority process temporarily inherits high priority process's priority

- Prevents medium priority processes from interfering

Priority Scheduling Advantages & Disadvantages:

****Advantages****:

- ****Flexibility****: Can assign priorities based on process importance

- ****System responsiveness****: Critical processes get immediate attention

- ****Real-time support****: Hard real-time systems use priority scheduling

****Disadvantages****:

- ****Starvation****: Low priority processes may never execute

- ****Priority inversion****: High priority blocked by low priority

- ****Complex priority assignment****: Difficult to determine optimal priorities

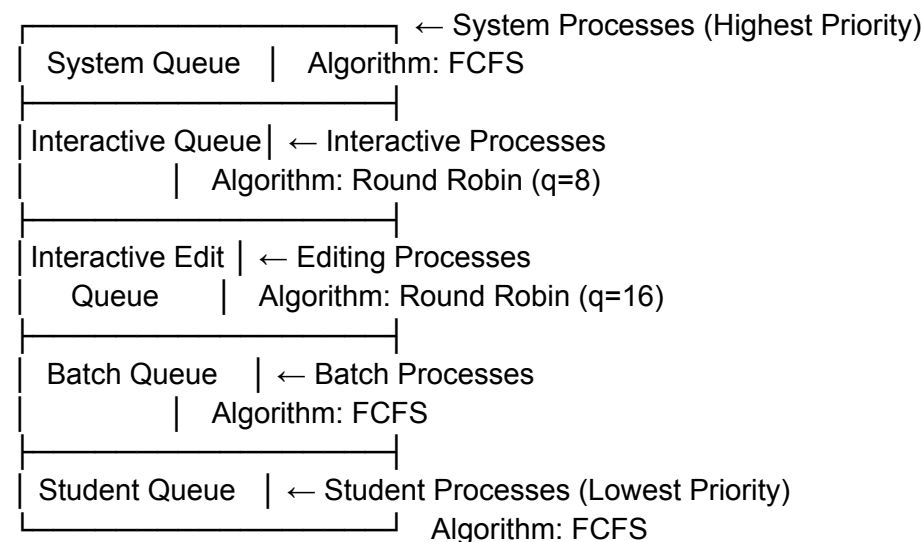
5. Multilevel Queue Scheduling {#multilevel-queue}

Algorithm Understanding:

Multilevel Queue is like ****different service counters at a bank**** - VIP customers, business customers, and regular customers have separate queues with different service policies.

Structure:

...



...

Queue Priority and Time Allocation:

- ****System Queue****: 40% CPU time
- ****Interactive Queue****: 30% CPU time
- ****Interactive Edit****: 20% CPU time
- ****Batch Queue****: 8% CPU time
- ****Student Queue****: 2% CPU time

Real-World Examples:

- ****Operating System****: Different process types in different queues
- ****Airport Security****: First class, business class, economy class lines
- ****Hospital****: Emergency, urgent care, routine checkup queues

🔥 Problem 1: Basic Multilevel Queue

****Given****:

Process	Arrival Time	Burst Time	Queue Type
P1	0	8	Batch
P2	1	2	System
P3	2	4	Interactive
P4	3	1	System
P5	4	3	Interactive

****Queue Configuration****:

- ****System Queue****: Highest priority, FCFS
- ****Interactive Queue****: Medium priority, RR(q=2)
- ****Batch Queue****: Lowest priority, FCFS

****Step-by-Step Solution**:**

****Queue Assignment**:**

- ****System****: P2, P4
- ****Interactive****: P3, P5
- ****Batch****: P1

****Execution Timeline**:**

****Time 0****: P1 starts (only process available)

****Time 1****: P2 arrives in System queue → Preempt P1, execute P2

****Time 2****: P3 arrives in Interactive queue, P2 continues (System has priority)

****Time 3****: P2 completes, P4 arrives in System queue → Execute P4

****Time 4****: P4 completes, P5 arrives in Interactive queue

Now execute Interactive queue with RR(q=2): P3, P5

****Time 4-6****: Execute P3 for 2 units (remaining=2)

****Time 6-8****: Execute P5 for 2 units (remaining=1)

****Time 8-10****: Execute P3 for 2 units (completes)

****Time 10-11****: Execute P5 for 1 unit (completes)

****Time 11****: Execute P1 (remaining=7 units)

****Time 18****: P1 completes

****Gantt Chart****:

...

|P1|P2(2)|P4(1)|P3|P5|P3|P5|--P1(7)--|

0 1 3 4 6 8 10 11 18

...

****Calculations****:

| Process | Queue | AT | BT | CT | TAT | WT |

|-----|-----|----|----|----|----|----|

| P1 | Batch | 0 | 8 | 18 | 18 | 10 |

| P2 | System | 1 | 2 | 3 | 2 | 0 |

| P3 | Inter. | 2 | 4 | 10 | 8 | 4 |

| P4 | System | 3 | 1 | 4 | 1 | 0 |

| P5 | Inter. | 4 | 3 | 11 | 7 | 4 |

****Average WT**** = $(10 + 0 + 4 + 0 + 4) / 5 = 3.6$

🔥 Problem 2: Multilevel Queue with Time Slicing

****Given****: Same processes, but with time slicing between queues

****Time Allocation****:

- System: 50% (5 units out of every 10)
- Interactive: 30% (3 units out of every 10)
- Batch: 20% (2 units out of every 10)

****Execution Pattern**** (10-unit cycles):

...

Cycle 1 (0-10): System(5), Interactive(3), Batch(2)

Cycle 2 (10-20): System(5), Interactive(3), Batch(2)

...and so on

...

****Step-by-Step Solution****:

****Time 0-5****: System queue gets 5 units

- P1 executes for 1 unit (preempted when P2 arrives)
- P2 executes for 2 units (completes)
- P4 executes for 1 unit (completes)
- 1 unit unused

****Time 5-8****: Interactive queue gets 3 units

- P3 executes for 2 units (RR quantum)
- P5 executes for 1 unit

****Time 8-10****: Batch queue gets 2 units

- P1 executes for 2 units

****Continue pattern until all processes complete... ****

🔥 Problem 3: Multilevel Queue Design Problem

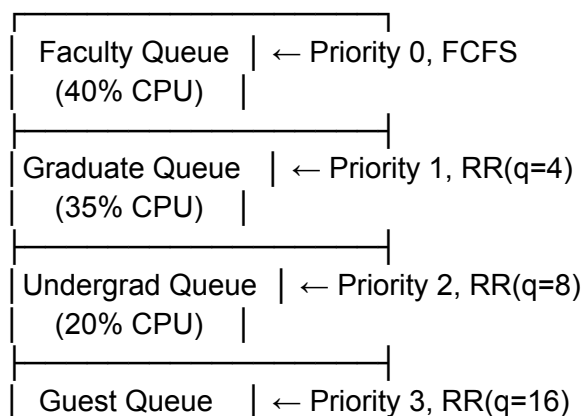
****Design Challenge****: Create optimal queue structure for university computer lab

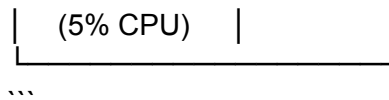
****Requirements****:

- Faculty processes: Highest priority
- Graduate student processes: High priority
- Undergraduate processes: Medium priority
- Guest processes: Lowest priority

****Proposed Design****:

...





Justification:

- **Faculty**: FCFS ensures their jobs complete quickly
- **Graduate**: Short quantum for responsiveness
- **Undergraduate**: Longer quantum, less overhead
- **Guest**: Longest quantum, minimal CPU allocation

Multilevel Queue Advantages & Disadvantages:

Advantages:

- **Flexibility**: Different algorithms for different process types
- **Priority separation**: Critical processes get guaranteed resources
- **Predictable performance**: Each queue gets fixed CPU allocation

Disadvantages:

- **Starvation**: Lower priority queues may not get CPU
- **Inflexible**: Processes can't move between queues
- **Complex scheduling**: Multiple algorithms to manage

6. Multilevel Feedback Queue {#multilevel-feedback}

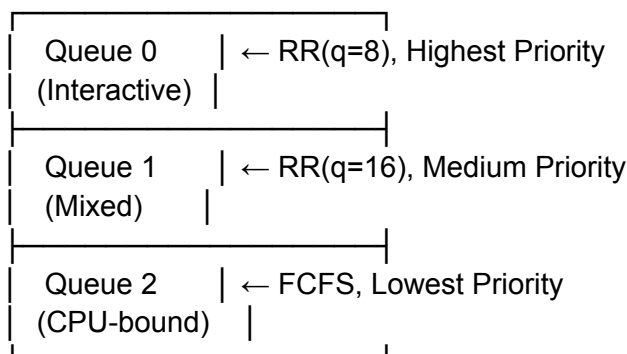
Algorithm Understanding:

Multilevel Feedback Queue is like **a school system** - students can move up or down grades based on their performance. CPU-bound processes move to lower priority, I/O-bound processes stay at higher priority.

Key Features:

1. **Process Migration**: Processes can move between queues
2. **Adaptive Priority**: Priority changes based on process behavior
3. **Aging Prevention**: Prevents starvation through promotion

Typical Configuration:



...

Movement Rules:

- **New processes**: Enter Queue 0
- **Quantum expiry**: Move to lower queue
- **I/O completion**: Move to higher queue
- **Aging**: Promote processes after waiting too long

Real-World Examples:

- **Windows Task Manager**: Background processes get lower priority over time
- **Unix nice values**: Long-running processes get "nicer" (lower priority)
- **Interactive systems**: Favor processes that interact with users

🔥 Problem 1: Basic Multilevel Feedback Queue

Given:

Process	Arrival Time	Burst Time	I/O Time
P1	0	12	None
P2	2	4	After 2
P3	4	8	After 3

Queue Configuration:

- **Queue 0**: RR(q=4), Priority 0
- **Queue 1**: RR(q=8), Priority 1
- **Queue 2**: FCFS, Priority 2

Movement Rules:

- Quantum expires → Move to next lower queue
- I/O completion → Move to Queue 0
- New process → Starts in Queue 0

Step-by-Step Solution:

Time 0: P1 enters Queue 0

Time 0-4: P1 executes for 4 units (quantum expires)
P1 moves to Queue 1, remaining time = 8

Time 4: P2 arrives in Queue 0, P3 arrives in Queue 0
Queue 0: [P2, P3], Queue 1: [P1]

Time 4-6: P2 executes for 2 units, then I/O (moves to I/O)

Time 6-10: P3 executes for 4 units (quantum expires)

P3 moves to Queue 1, remaining time = 4

Queue 1: [P1, P3]

Time 10: P2 completes I/O, returns to Queue 0

Queue 0: [P2], Queue 1: [P1, P3]

****Time 10-12**:** P2 executes for 2 units (completes)
****Time 12-20**:** P1 executes for 8 units (completes)
****Time 20-24**:** P3 executes for 4 units (completes)

****Gantt Chart**:**

```

...
|P1(4)|P2|P3(4)|P2|P1(8)|P3(4)|
0  4 6   10 12  20  24
...

```

****Process Movement Tracking**:**

```

...
Time | P1 Queue | P2 Queue | P3 Queue | Action
0   | 0       | -       | -       | P1 starts
4   | 1↓     | 0       | 0       | P1 demoted, P2&P3 arrive
6   | 1       | I/O     | 0       | P2 to I/O
10  | 1       | 0↑     | 1↓     | P2 promoted, P3 demoted
...

```

🔥 Problem 2: Multilevel Feedback with Aging

****Given**:** Same processes, but with aging mechanism

****Aging Rule**:** If process waits in lower queue for >10 time units, promote to higher queue

****Modified Execution**:**

****Time 0-4**:** P1 in Queue 0 (executes 4, moves to Queue 1)
****Time 4-10**:** P3 executes in Queue 0, P1 waits in Queue 1
****Time 10**:** P1 waited 6 units in Queue 1 (no aging yet)
****Time 12**:** P2 completes, P1 executes in Queue 1
****Time 20**:** P1 completes
****Time 20**:** P3 in Queue 1, executes remaining 4 units

****Time 14**:** If P1 was still waiting, it would be promoted back to Queue 0 after waiting 10 units

🔥 Problem 3: Multilevel Feedback Queue Comparison

****Compare MLFQ vs Fixed Priority for same process set**:**

****Processes**:**

Process	Type	Burst Time	Behavior
P1	CPU-bound	20	No I/O
P2	Interactive	8	Frequent I/O
P3	Mixed	12	Some I/O

****MLFQ Result**:**

- P2 stays in high-priority queue (good response time)
- P1 moves to low-priority queue (doesn't block others)
- P3 fluctuates between queues based on behavior

****Fixed Priority Result**:**

- Priority assignment is static
- Poor adaptation to process behavior changes
- May cause starvation if priorities poorly assigned

****Performance Comparison**:**

Metric	MLFQ	Fixed Priority
Response Time	Better for interactive	Depends on assignment
Adaptability	High	Low
Fairness	Good	Poor if misassigned
Overhead	Higher	Lower

MLFQ Parameters Tuning:

1. ****Number of Queues****: More queues = finer granularity
2. ****Time Quanta****: Shorter at top, longer at bottom
3. ****Promotion Rules****: When to move processes up
4. ****Demotion Rules****: When to move processes down

****Example Configuration**:**

...

Queue 0: RR(q=2) ← Very responsive
Queue 1: RR(q=4) ← Moderate response
Queue 2: RR(q=8) ← Less responsive
Queue 3: FCFS ← Batch processing

...

Multilevel Feedback Queue Advantages & Disadvantages:

****Advantages**:**

- ****Adaptive****: Automatically adjusts to process behavior
- ****Fair****: Prevents starvation through aging
- ****Responsive****: Good for interactive processes
- ****Efficient****: CPU-bound processes don't block I/O-bound processes

****Disadvantages**:**

- ****Complex****: Many parameters to tune
- ****Overhead****: Process migration and queue management
- ****Unpredictable****: Process priority changes dynamically
- ****Starvation possible****: Without proper aging mechanism

Comparison and Selection Criteria {#comparison}

Algorithm Comparison Table:

Algorithm	Preemptive	Starvation	Complexity	Best Use Case
FCFS	No	No	Low	Batch systems
SJF	Optional	Yes (long jobs)	Medium	Known job times
Round Robin	Yes	No	Medium	Time-sharing
Priority	Optional	Yes (low priority)	Medium	Real-time systems
Multilevel Queue	Yes	Possible	High	Mixed workloads
MLFQ	Yes	No (with aging)	Very High	General purpose

Performance Metrics Comparison:

For typical workload mix:

Algorithm	Avg Response Time	Avg Waiting Time	Throughput	Fairness
FCFS	Poor	Poor	Low	Fair
SJF	Excellent	Excellent	High	Unfair
Round Robin	Good	Fair	Medium	Excellent
Priority	Variable	Variable	High	Poor
MLFQ	Good	Good	High	Good

Selection Guidelines:

- **Batch Systems**:** FCFS or SJF
 - Predictable workload
 - No user interaction required
 - Throughput more important than response time
- **Interactive Systems**:** Round Robin or MLFQ
 - User responsiveness critical
 - Fair sharing among users
 - Variable workload types
- **Real-time Systems**:** Priority Scheduling
 - Deadline constraints
 - Predictable response times
 - Critical task priorities
- **General Purpose OS**:** MLFQ
 - Mixed workload (interactive + batch)
 - Adaptive to changing conditions
 - Good overall performance

Exam Strategy Tips:

1. **Algorithm Identification**:
 - Read problem constraints carefully
 - Identify if preemption is allowed
 - Check for arrival times vs. burst times
2. **Calculation Steps**:
 - Draw Gantt chart first
 - Calculate completion times
 - Use formulas: $TAT = CT - AT$, $WT = TAT - BT$
3. **Comparison Questions**:
 - Focus on key differences
 - Use specific examples
 - Consider trade-offs
4. **Design Questions**:
 - Justify algorithm choice
 - Consider workload characteristics
 - Address performance requirements

Common Exam Mistakes:

1. **FCFS**: Forgetting to sort by arrival time
2. **SJF**: Confusing preemptive vs non-preemptive
3. **Round Robin**: Incorrect queue management
4. **Priority**: Missing preemption opportunities
5. **General**: Calculation errors in TAT/WT formulas

Practice Problem Categories:

1. **Basic Calculations**: Given processes, calculate metrics
2. **Algorithm Comparison**: Same processes, different algorithms
3. **Design Problems**: Choose algorithm for given scenario
4. **Hybrid Problems**: Multiple algorithms in sequence
5. **Optimization**: Tune parameters for best performance

This comprehensive guide provides step-by-step understanding of each scheduling algorithm with multiple solved problems, real-world context, and exam-focused explanations. Each algorithm section includes detailed problem solutions that demonstrate both the mechanical steps and conceptual understanding needed for examinations.

4 | 2 |

| P4 | 3 | 5 | 3 | 10 | 7 | 2 | 2 |

Average TAT = $(4 + 12 + 3 + 7) / 4 = 6.5$

Average WT = $(0 + 9 + 2 + 2) / 4 = 3.25$

Preemptive Priority Algorithm:

...

PriorityScheduling_Preemptive(processes[]):

1. current_time = 0
2. While processes remain:
 - a. Check for new arrivals at current_time
 - b. If new arrival has higher priority than current process:
Preempt current process
 - c. Execute highest priority process for 1 time unit
 - d. Update current_time and remaining times

...

🔥 Problem 2: Preemptive Priority

****Given**** (Same data as Problem 1):

| Process | Arrival Time | Burst Time | Priority |

P1	0	4	2	
P2	1	3	1	
P3	2	1	4	
P4	3	5	3	

****Step-by-Step Solution****:

****Time-by-Time Analysis****:

****t=0****: P1 starts (Priority=2)

****t=1****: P2 arrives (Priority=1). $P1(2) > P2(1) \rightarrow$ Continue P1

****t=2****: P3 arrives (Priority=4). $P3(4) > P1(2) \rightarrow$ Preempt P1, start P3

****t=3****: P3 continues, P4 arrives (Priority=3). $P3(4) > P4(3) \rightarrow$ Continue P3

****t=3****: P3 completes. Available: P1(Priority=2, RT=2), P2(Priority=1), P4(Priority=3)

Highest priority: P4(3) $\rightarrow$ Start P4

****t=8****: P4 completes. Available: P1(Priority=2), P2(Priority=1)

Highest priority: P1(2) $\rightarrow$ Resume P1

****t=10****: P1 completes. Available: P2(Priority=1)

Execute P2

****Gantt Chart****:

...

|P1|P3(1)|--P4(5)--|P1|P2(3)|

0 2 3 8 10 13

...

****Calculations****:

| Process | AT | BT | Priority | CT | TAT | WT | RT |

-----	-----	-----	-----	-----	-----	-----	-----
P1	0	4	2	10	10	6	0

P2	1	3	1	13	12	9	9	
P3	2	1						